VittaCare

Banco de Dados – Bloco 5

Gabriela Dias Magalhães

Sumário

1. Introdução 2
2.Decisão Tecnológica do Banco de Dados 2
2.1 Justificativa técnica 3
2.2 Decisão registrada 4
3. Modelo conceitual (ER) 4
3.1 Entidades do domínio 5
3.2 Relacionamentos e cardinalidades 6
3.3 Representação textual do ER 7
4. Modelo lógico 7
4.1 Convenções 8
4.2 Estrutura lógica resumida 8
4.3 Índices e justificativas 10
5. Modelo físico — SQL executável 10
5.1 Regras de execução e concorrência 13
6. Dicionário de dados 14
clinica 14
usuario 14
paciente 15
profissional 16
especialidade 16
profissional_especialidade 16
disponibilidade 17
agendamento 17
atendimento 18
notificacao 19
papel / permissao / usuario_papel / papel_permissao 19
registro_auditoria 19
7. Estratégia de dados: backup, LGPD e migrations 20
7.1 Backup e recuperação 20
7.2 LGPD e proteção de dados 21

7.3 Migrations e evolução do esquema 23
7.4 Estratégia para migrations sem downtime 23
8. Regras de integridade e rastreabilidade 24
8.1 Pontos que exigem validação na implementação..... 25
9. Considerações finais 26

1. Introdução

Este documento apresenta a modelagem de dados do VittaCare, plataforma web SaaS destinada ao gerenciamento de clínicas e ambulatórios. O objetivo é transformar as entidades e regras identificadas nos documentos de concepção, requisitos, arquitetura e UML em uma estrutura relacional consistente, segura e implementável.

A modelagem contempla pacientes, usuários, profissionais de saúde, especialidades, disponibilidades, agendamentos, atendimentos, notificações e recursos administrativos. O sistema não armazenará prontuários eletrônicos completos, prescrições, diagnósticos automatizados nem dados de pagamento, pois essas funcionalidades estão fora do escopo definido para a versão atual.

A proposta adota banco compartilhado com isolamento lógico por clínica, utilizando a coluna clinica_id nas tabelas que pertencem a uma instituição. O contexto da clínica deverá ser obtido a partir da identidade autenticada no back-end; um clinica_id enviado pelo front-end nunca será considerado prova de autorização.

O modelo foi elaborado para atender, principalmente, aos requisitos RF001–RF026, com destaque para autenticação, gerenciamento de cadastros, disponibilidade, prevenção de agendamentos duplicados, registro de comparecimento, permissões e isolamento entre clínicas.

2.Decisão Tecnológica do Banco de Dados

A escolha do banco deve acompanhar a arquitetura definida no Bloco 3. Como o VittaCare utiliza Java com Spring Boot, Spring Data JPA/Hibernate e uma arquitetura de monólito modular, foi selecionado um banco relacional compatível com transações, chaves estrangeiras, restrições de integridade e consultas relacionais.

Critério	PostgreSQL — escolhido	MongoDB — alternativa descartada	MySQL — alternativa avaliada
Modelo de dados	Relacional, adequado às entidades e associações do domínio.	Documento, exigiria tratamento adicional para relações e integridade entre documentos.	Relacional e tecnicamente viável.
Integridade	Suporte a PK, FK, UNIQUE, CHECK, transações e índices.	Não oferece o mesmo conjunto de constraints relacionais para este modelo.	Suporta PK, FK, constraints e transações.
Agendamentos	Permite transações e índices únicos para reduzir conflitos concorrentes.	Regras de integridade entre documentos exigiriam maior implementação na aplicação.	Também atende às operações transacionais.
Multi-tenant	Permite FKs compostas e constraints envolvendo clinica_id.	O isolamento dependeria mais de validações implementadas pela aplicação.	Também permite isolamento lógico relacional.
Decisão	Selecionado por aderência direta à arquitetura e às regras do domínio.	Descartado por menor aderência ao conjunto de relações e constraints exigidas.	Não oferece índice único parcial nativo, necessário para impedir mais de um agendamento ativo por disponibilidade

2.1 Justificativa técnica

- O domínio possui relações explícitas entre clínicas, usuários, pacientes, profissionais, especialidades, disponibilidades e agendamentos.
- A confirmação de uma consulta exige consistência transacional e prevenção de duas marcações ativas para o mesmo horário.
- O isolamento lógico por clínica pode ser reforçado por chaves estrangeiras compostas que associam os registros ao mesmo clinica_id.
- PostgreSQL é compatível com Spring Data JPA/Hibernate e com a hospedagem prevista no Amazon RDS Multi-AZ.

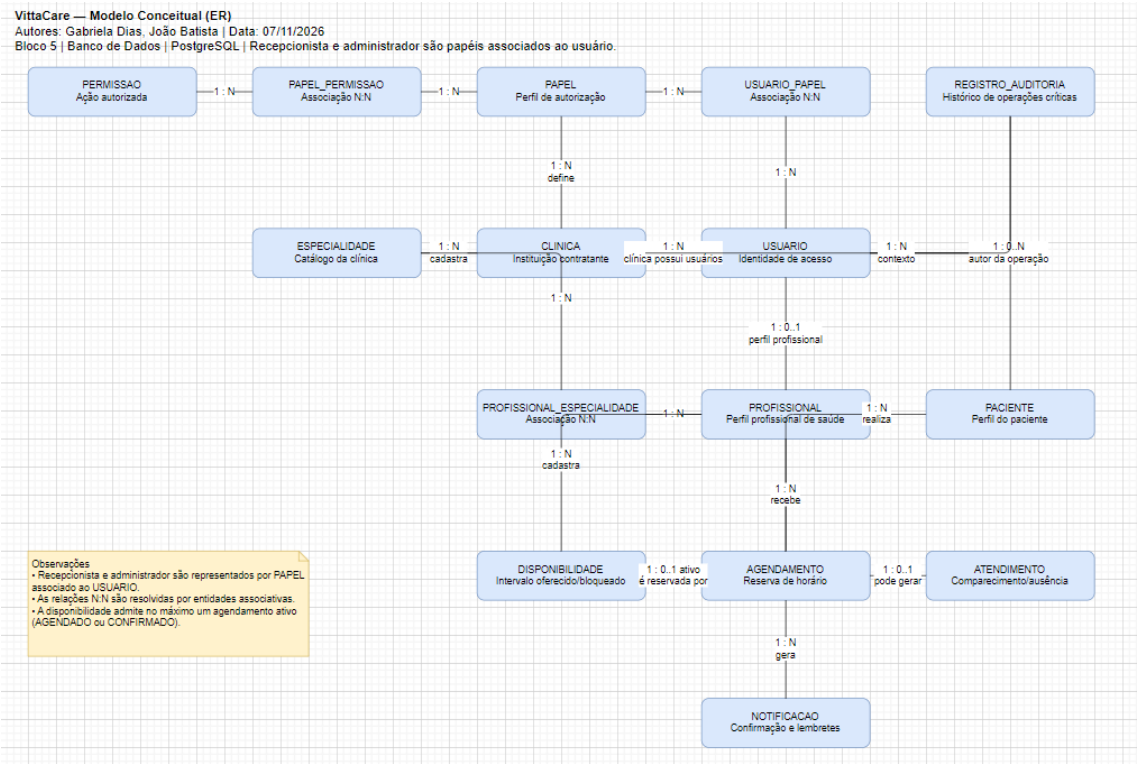
Alternativas: MongoDB foi descartado por não oferecer o mesmo modelo relacional e conjunto de constraints que atendem diretamente às relações e à integridade do domínio. MySQL foi considerado, mas não apresentou vantagem suficiente para substituir a tecnologia já definida no Bloco 3.

2.2 Decisão registrada

Item	Definição
Banco de dados	PostgreSQL
Hospedagem	Amazon RDS, configuração Multi-AZ
Persistência na aplicação	Spring Data JPA / Hibernate
Modelo de tenancy	Banco compartilhado com isolamento lógico por clinica_id
Estratégia de integridade	PKs, FKs, FKs compostas, UNIQUE, CHECK e transações
Responsável pelas regras	Back-end Spring Boot; o banco reforça as regras por constraints

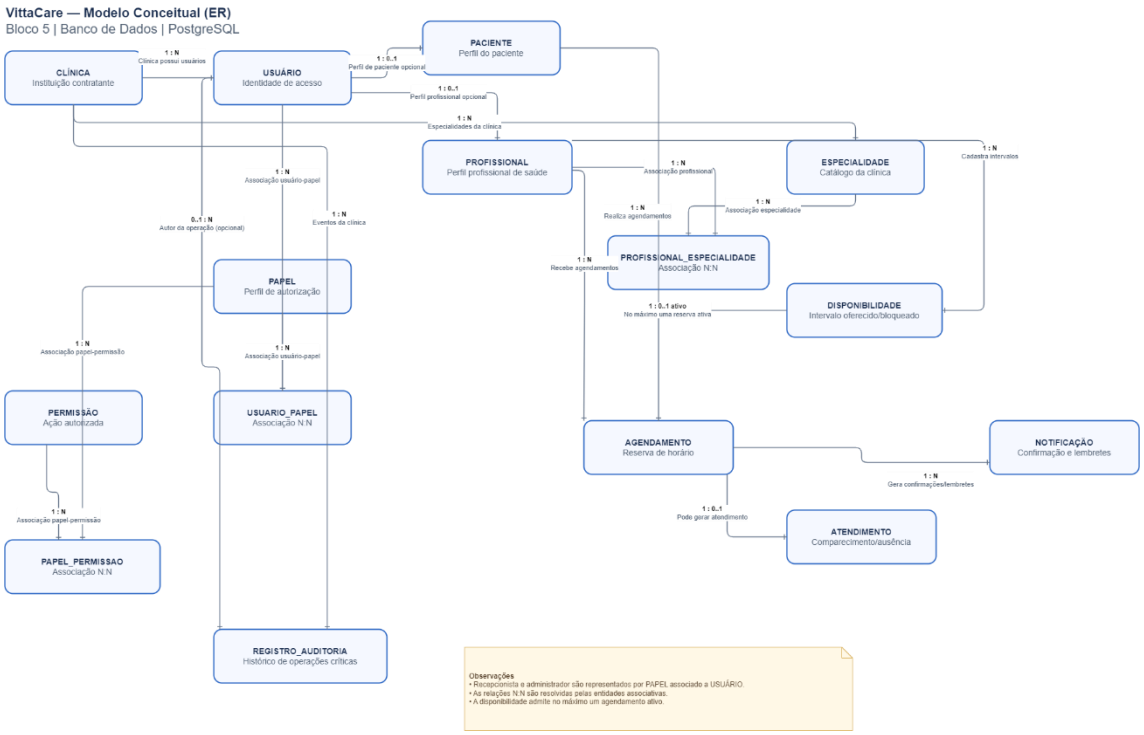
3. Modelo conceitual (ER)

O modelo conceitual identifica as entidades do domínio e seus relacionamentos, sem se prender inicialmente aos detalhes de implementação. As cardinalidades representam as regras previstas para a versão atual do VittaCare.



3.1 Entidades do domínio

Entidade	Descrição de negócio
Clínica	Instituição contratante que delimita o conjunto de dados acessível no sistema.
Usuário	Identidade de acesso, com e-mail, senha protegida, situação da conta e vínculo com a clínica.
Paciente	Perfil de paciente associado a uma identidade de usuário.
Profissional de saúde	Perfil profissional responsável por disponibilidades e consultas.
Especialidade	Especialidade médica cadastrada para uma clínica.
Profissional_Especialidade	Associação entre profissionais e suas especialidades.
Disponibilidade	Intervalo de horário oferecido ou bloqueado por um profissional.
Agendamento	Reserva de uma disponibilidade para um paciente.
Atendimento	Registro de comparecimento ou ausência vinculado ao agendamento.
Notificação	Confirmação ou lembrete relacionado a um agendamento.
Papel e Permissão	Estrutura de autorização para diferenciar paciente, profissional, recepcionista e administrador.
Registro de auditoria	Histórico de operações administrativas e críticas.



3.2 Relacionamentos e cardinalidades

Relacionamento	Cardinalidade	Regra
Clínica — Usuário	1 : N	Uma clínica possui vários usuários; cada usuário pertence a uma clínica na versão inicial.
Usuário — Paciente	1 : 0..1	Um usuário pode possuir um perfil de paciente.
Usuário — Profissional	1 : 0..1	Um usuário pode possuir um perfil profissional.
Clínica — Especialidade	1 : N	Cada especialidade cadastrada pertence a uma clínica.
Profissional — Especialidade	N : N	Um profissional pode possuir várias especialidades e uma especialidade pode estar associada a vários profissionais.
Profissional — Disponibilidade	1 : N	Um profissional cadastra vários intervalos de disponibilidade.
Paciente — Agendamento	1 : N	Um paciente pode realizar vários agendamentos.

Profissional — Agendamento	1 : N	Um profissional pode receber vários agendamentos em horários distintos.
Disponibilidade — Agendamento	1 : 0..1 ativo	Uma disponibilidade pode ter no máximo um agendamento ativo.
Agendamento — Atendimento	1 : 0..1	Um agendamento pode gerar um registro de comparecimento ou ausência.
Agendamento — Notificação	1 : N	Um agendamento pode gerar confirmação e lembretes.
Usuário — Papel	N : N	Um usuário pode receber um ou mais papéis, conforme autorização administrativa.
Papel — Permissão	N : N	Cada papel reúne as permissões que pode executar.

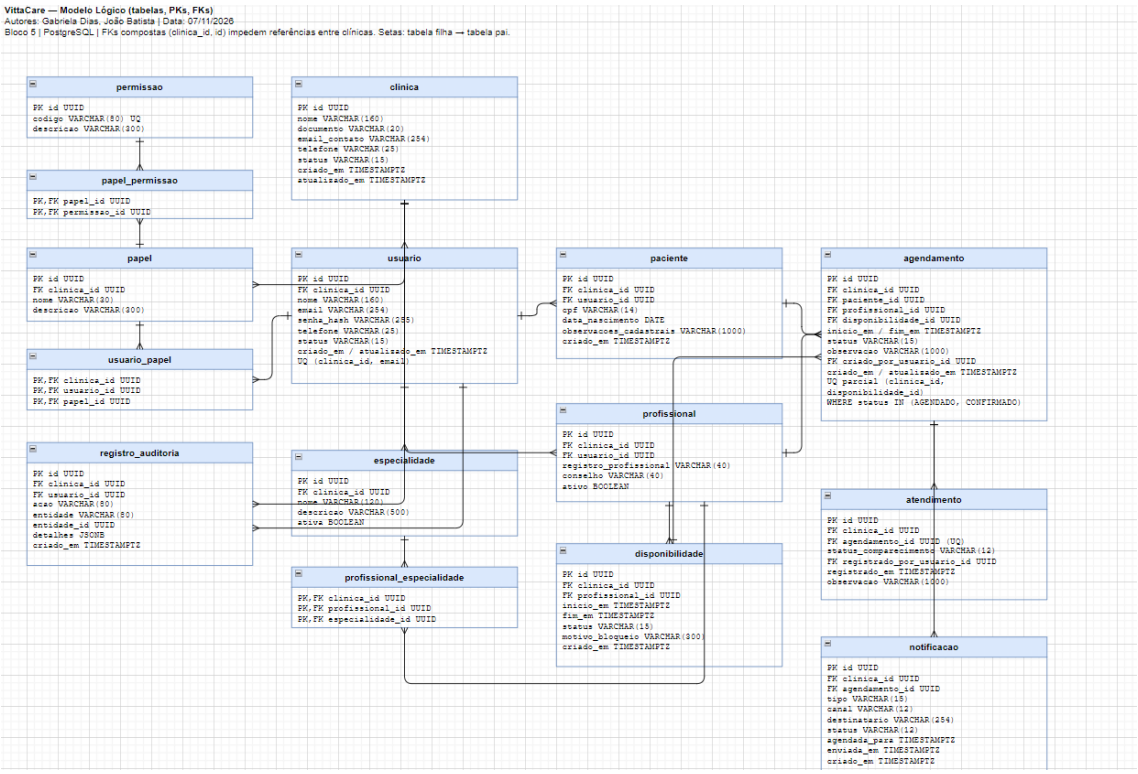
3.3 Representação textual do ER

CLINICA 1——N USUARIO; USUARIO 1——0..1 PACIENTE; USUARIO 1——0..1 PROFISSIONAL; CLINICA 1——N ESPECIALIDADE; PROFISSIONAL N——N ESPECIALIDADE (por PROFISSIONAL_ESPECIALIDADE); PROFISSIONAL 1——N DISPONIBILIDADE; PACIENTE 1——N AGENDAMENTO; PROFISSIONAL 1——N AGENDAMENTO; DISPONIBILIDADE 1——0..1 AGENDAMENTO ATIVO; AGENDAMENTO 1——0..1 ATENDIMENTO; AGENDAMENTO 1——N NOTIFICACAO; USUARIO N——N PAPEL (por USUARIO_PAPEL); PAPEL N——N PERMISSAO (por PAPEL_PERMISSAO).

Observação de mapeamento UML: recepcionista e administrador são representados como papéis associados ao usuário, em vez de tabelas com dados duplicados. Essa decisão mantém os atributos comuns de autenticação em USUARIO e concentra as diferenças de acesso na estrutura de autorização.

4. Modelo lógico

O modelo lógico converte as entidades em tabelas relacionais, definindo atributos, tipos, chaves primárias (PK), chaves estrangeiras (FK), restrições e índices. Os identificadores são UUID para facilitar a geração de IDs pela aplicação Java e evitar dependência de sequências numéricas distribuídas.



4.1 Convenções

• Todos os identificadores principais utilizam UUID.

• Tabelas pertencentes a uma clínica possuem clinica_id e, quando aplicável, FKs compostas (clinica_id, id) para impedir referências cruzadas entre instituições.

• Datas e horários são armazenados com TIMESTAMPTZ, mantendo referência de fuso horário.

• Valores monetários não fazem parte do escopo atual do sistema.

• Senhas são armazenadas somente como hash; nunca em texto puro.

• Campos de situação utilizam VARCHAR com CHECK para restringir os valores permitidos.

• O modelo lógico está normalizado até a Terceira Forma Normal (3FN): cada tabela descreve uma única entidade, os atributos dependem somente da chave primária e os dados que se repetem (especialidade, papel e permissão) ficam em tabelas próprias. Evitou-se a desnormalização para preservar a integridade e reduzir anomalias de atualização.

4.2 Estrutura lógica resumida

Tabela	PK	Principais FKs	Finalidade
clinica	id	—	Organização contratante.

usuario	id	clinica_id → clinica	Identidade, autenticação e estado da conta.
paciente	id	(clinica_id, usuario_id) → usuario	Dados cadastrais do paciente.
profissional	id	(clinica_id, usuario_id) → usuario	Dados profissionais.
especialidade	id	clinica_id → clinica	Catálogo de especialidades por clínica.
profissional_especialidade	(profissional_id, especialidade_id)	FKs compostas para profissional e especialidade	Associação N:N.
disponibilidade	id	(clinica_id, profissional_id) → profissional	Intervalos disponíveis ou bloqueados.
agendamento	id	FKs para clínica, paciente, profissional e disponibilidade	Reserva e situação da consulta.
atendimento	id	(clinica_id, agendamento_id) → agendamento	Comparecimento ou ausência.
notificacao	id	(clinica_id, agendamento_id) → agendamento	Confirmações e lembretes.
papel	id	clinica_id → clinica	Papéis de acesso.
permissao	id	—	Permissões cadastradas.
usuario_papel	(usuario_id, papel_id)	FKs compostas	Papéis atribuídos a usuários.
papel_permissao	(papel_id, permissao_id)	FKs	Permissões de cada papel.
registro_auditoria	id	FKs para clínica e usuário	Rastreabilidade de operações críticas.

4.3 Índices e justificativas

Índice	Justificativa
usuario (clinica_id, email)	Acelera autenticação e impede e-mails duplicados dentro da mesma clínica.
profissional_especialidade (clinica_id, especialidade_id)	Apoia a pesquisa de profissionais por especialidade.
disponibilidade (clinica_id, profissional_id, inicio_em)	Apoia a consulta de horários de um profissional em ordem cronológica.
agendamento (clinica_id, paciente_id, inicio_em)	Apoia o histórico de consultas do paciente.
agendamento (clinica_id, profissional_id, inicio_em)	Apoia a agenda do profissional.
Índice único parcial em agendamento(disponibilidade_id)	Impede mais de um agendamento ativo para a mesma disponibilidade.
notificacao (clinica_id, agendamento_id, status)	Apoia o processamento de notificações pendentes.
registro_auditoria (clinica_id, criado_em)	Apoia consultas de auditoria por clínica e período.

5. Modelo físico — SQL executável

O script a seguir utiliza sintaxe PostgreSQL. Ele cria as tabelas principais, chaves, restrições e índices. A aplicação deverá gerar os UUIDs e executar as operações por meio do Spring Data JPA/Hibernate ou migrations versionadas.

```
CREATE TABLE clinica (
  id UUID PRIMARY KEY,
  nome VARCHAR(160) NOT NULL,
  documento VARCHAR(20),
  email_contato VARCHAR(254),
  telefone VARCHAR(25),
  status VARCHAR(15) NOT NULL DEFAULT 'ATIVA'
    CHECK (status IN ('ATIVA', 'INATIVA')),
  criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  atualizado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE usuario (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL REFERENCES clinica(id) ON DELETE RESTRICT,
  nome VARCHAR(160) NOT NULL,
  email VARCHAR(254) NOT NULL,
  senha_hash VARCHAR(255) NOT NULL,
  telefone VARCHAR(25),
  status VARCHAR(15) NOT NULL DEFAULT 'ATIVO'
    CHECK (status IN ('ATIVO', 'INATIVO', 'BLOQUEADO')),
  criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  atualizado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uq_usuario_clinica_id UNIQUE (clinica_id, id),
  CONSTRAINT uq_usuario_email_clinica UNIQUE (clinica_id, email)
);

CREATE TABLE paciente (
 id UUID PRIMARY KEY,
```

```
 clinica_id UUID NOT NULL,
 usuario_id UUID NOT NULL,
 cpf VARCHAR(14),
 data_nascimento DATE,
 observacoes_cadastrais VARCHAR(1000),
 criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT uq_paciente_clinica_id UNIQUE (clinica_id, id),
 CONSTRAINT uq_paciente_usuario UNIQUE (clinica_id, usuario_id),
 CONSTRAINT uq_paciente_cpf UNIQUE (clinica_id, cpf),
 CONSTRAINT fk_paciente_usuario
 FOREIGN KEY (clinica_id, usuario_id)
 REFERENCES usuario (clinica_id, id) ON DELETE RESTRICT
);

CREATE TABLE profissional (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL,
  usuario_id UUID NOT NULL,
  registro_profissional VARCHAR(40),
  conselho VARCHAR(40),
  ativo BOOLEAN NOT NULL DEFAULT TRUE,
  criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uq_profissional_clinica_id UNIQUE (clinica_id, id),
  CONSTRAINT uq_profissional_usuario UNIQUE (clinica_id, usuario_id),
  CONSTRAINT fk_profissional_usuario
    FOREIGN KEY (clinica_id, usuario_id)
    REFERENCES usuario (clinica_id, id) ON DELETE RESTRICT
);

CREATE TABLE especialidade (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL REFERENCES clinica(id) ON DELETE RESTRICT,
  nome VARCHAR(120) NOT NULL,
  descricao VARCHAR(500),
  ativa BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT uq_especialidade_clinica_id UNIQUE (clinica_id, id),
  CONSTRAINT uq_especialidade_nome UNIQUE (clinica_id, nome)
);

CREATE TABLE profissional_especialidade (
  clinica_id UUID NOT NULL,
  profissional_id UUID NOT NULL,
  especialidade_id UUID NOT NULL,
  PRIMARY KEY (clinica_id, profissional_id, especialidade_id),
  FOREIGN KEY (clinica_id, profissional_id)
    REFERENCES profissional (clinica_id, id) ON DELETE RESTRICT,
  FOREIGN KEY (clinica_id, especialidade_id)
    REFERENCES especialidade (clinica_id, id) ON DELETE RESTRICT
);

CREATE TABLE disponibilidade (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL,
  profissional_id UUID NOT NULL,
  inicio_em TIMESTAMPTZ NOT NULL,
  fim_em TIMESTAMPTZ NOT NULL,
  status VARCHAR(15) NOT NULL DEFAULT 'DISPONIVEL'
    CHECK (status IN ('DISPONIVEL', 'BLOQUEADO', 'AGENDADO')),
  motivo_bloqueio VARCHAR(300),
  criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT ck_disponibilidade_periodo CHECK (fim_em > inicio_em),
  CONSTRAINT uq_disponibilidade_clinica_id UNIQUE (clinica_id, id),
  CONSTRAINT fk_disponibilidade_profissional
    FOREIGN KEY (clinica_id, profissional_id)
    REFERENCES profissional (clinica_id, id) ON DELETE RESTRICT
);

CREATE TABLE agendamento (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL,
  paciente_id UUID NOT NULL,
  profissional_id UUID NOT NULL,
  disponibilidade_id UUID NOT NULL,
  inicio_em TIMESTAMPTZ NOT NULL,
  fim_em TIMESTAMPTZ NOT NULL,
  status VARCHAR(15) NOT NULL DEFAULT 'AGENDADO'
    CHECK (status IN ('AGENDADO', 'CONFIRMADO', 'CANCELADO', 'REALIZADO')),
  observacao VARCHAR(1000),
  criado_por_usuario_id UUID,
  criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  atualizado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
  CONSTRAINT ck_agendamento_periodo CHECK (fim_em > inicio_em),
  CONSTRAINT uq_agendamento_clinica_id UNIQUE (clinica_id, id),
  CONSTRAINT fk_agendamento_paciente
    FOREIGN KEY (clinica_id, paciente_id)
    REFERENCES paciente (clinica_id, id) ON DELETE RESTRICT,
  CONSTRAINT fk_agendamento_profissional
    FOREIGN KEY (clinica_id, profissional_id)
    REFERENCES profissional (clinica_id, id) ON DELETE RESTRICT,
  CONSTRAINT fk_agendamento_disponibilidade
    FOREIGN KEY (clinica_id, disponibilidade_id)
    REFERENCES disponibilidade (clinica_id, id) ON DELETE RESTRICT,
  CONSTRAINT fk_agendamento_criador
    FOREIGN KEY (clinica_id, criado_por_usuario_id)
    REFERENCES usuario (clinica_id, id) ON DELETE RESTRICT
);

-- Uma disponibilidade só pode possuir um agendamento ativo.
CREATE UNIQUE INDEX uq_agendamento_disponibilidade_ativa
  ON agendamento (clinica_id, disponibilidade_id)
  WHERE status IN ('AGENDADO', 'CONFIRMADO');

CREATE TABLE atendimento (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL,
  agendamento_id UUID NOT NULL,
  status_comparecimento VARCHAR(12) NOT NULL
    CHECK (status_comparecimento IN ('PRESENTE', 'AUSENTE')),
  registrado_por_usuario_id UUID NOT NULL,
  registrado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  observacao VARCHAR(1000),
  CONSTRAINT uq_atendimento_agendamento UNIQUE (clinica_id, agendamento_id),
  FOREIGN KEY (clinica_id, agendamento_id)
    REFERENCES agendamento (clinica_id, id) ON DELETE RESTRICT,
  FOREIGN KEY (clinica_id, registrado_por_usuario_id)
    REFERENCES usuario (clinica_id, id) ON DELETE RESTRICT
);

CREATE TABLE notificacao (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL,
  agendamento_id UUID NOT NULL,
  tipo VARCHAR(15) NOT NULL
    CHECK (tipo IN ('CONFIRMACAO', 'LEMBRETE')),
  canal VARCHAR(12) NOT NULL
    CHECK (canal IN ('EMAIL', 'SISTEMA')),
  destinatario VARCHAR(254) NOT NULL,
  status VARCHAR(12) NOT NULL DEFAULT 'PENDENTE'
    CHECK (status IN ('PENDENTE', 'ENVIADA', 'FALHA')),
  agendada_para TIMESTAMPTZ,
  enviada_em TIMESTAMPTZ,
  criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (clinica_id, agendamento_id)
    REFERENCES agendamento (clinica_id, id) ON DELETE RESTRICT
);

CREATE TABLE papel (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL REFERENCES clinica(id) ON DELETE RESTRICT,
  nome VARCHAR(30) NOT NULL
    CHECK (nome IN ('PACIENTE', 'PROFISSIONAL', 'RECEPCIONISTA', 'ADMINISTRADOR')),
  descricao VARCHAR(300),
  CONSTRAINT uq_papel_clinica_id UNIQUE (clinica_id, id),
  CONSTRAINT uq_papel_nome UNIQUE (clinica_id, nome)
);

CREATE TABLE permissao (
  id UUID PRIMARY KEY,
  codigo VARCHAR(80) NOT NULL UNIQUE,
  descricao VARCHAR(300) NOT NULL
);

CREATE TABLE usuario_papel (
  clinica_id UUID NOT NULL,
  usuario_id UUID NOT NULL,
  papel_id UUID NOT NULL,
  PRIMARY KEY (clinica_id, usuario_id, papel_id),
  FOREIGN KEY (clinica_id, usuario_id)
    REFERENCES usuario (clinica_id, id) ON DELETE RESTRICT,
  FOREIGN KEY (clinica_id, papel_id)
    REFERENCES papel (clinica_id, id) ON DELETE RESTRICT
);
```

```
CREATE TABLE papel_permissao (
  papel_id UUID NOT NULL REFERENCES papel(id) ON DELETE RESTRICT,
  permissao_id UUID NOT NULL REFERENCES permissao(id) ON DELETE RESTRICT,
  PRIMARY KEY (papel_id, permissao_id)
);

CREATE TABLE registro_auditoria (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL REFERENCES clinica(id) ON DELETE RESTRICT,
  usuario_id UUID,
  acao VARCHAR(80) NOT NULL,
  entidade VARCHAR(80) NOT NULL,
  entidade_id UUID,
  detalhes JSONB,
  criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (clinica_id, usuario_id)
    REFERENCES usuario (clinica_id, id) ON DELETE RESTRICT
);

-- Índices para as consultas mais frequentes.
CREATE INDEX ix_disponibilidade_profissional_inicio
  ON disponibilidade (clinica_id, profissional_id, inicio_em);

CREATE INDEX ix_agendamento_paciente_inicio
  ON agendamento (clinica_id, paciente_id, inicio_em DESC);

CREATE INDEX ix_agendamento_profissional_inicio
  ON agendamento (clinica_id, profissional_id, inicio_em);

CREATE INDEX ix_notificacao_pendente
  ON notificacao (clinica_id, status, agendada_para);

CREATE INDEX ix_auditoria_clinica_data
  ON registro_auditoria (clinica_id, criado_em DESC);
```

5.1 Regras de execução e concorrência

• A criação de agendamento deve ocorrer em transação. O serviço valida o contexto da clínica, consulta a disponibilidade e tenta inserir o registro.

• Ciclo de vida do agendamento: AGENDADO (reserva criada, disponibilidade ocupada) → CONFIRMADO (presença confirmada pelo paciente ou pela recepção, por exemplo após o lembrete) → REALIZADO (comparecimento ou ausência registrados na tabela atendimento). CANCELADO pode ocorrer a partir de AGENDADO ou CONFIRMADO. Os status AGENDADO e CONFIRMADO são considerados ativos: ocupam a disponibilidade (índice único parcial) e impedem seu bloqueio (RF026).

• O índice único parcial uq_agendamento_disponibilidade_ativa é a última barreira contra duas reservas ativas para a mesma disponibilidade. Se duas requisições concorrentes tentarem reservar o mesmo horário, uma delas deverá falhar por violação de unicidade.

• A alteração de status da disponibilidade e a criação ou cancelamento do agendamento devem ser realizadas de forma coordenada na mesma transação.

• Antes de bloquear um horário, o serviço deve verificar agendamentos ativos associados, na mesma transação do bloqueio. O banco impede duplicidade de reserva, mas a regra RF026 também precisa ser validada na camada de serviço.

• As FKs compostas garantem que paciente, profissional, disponibilidade, usuário e agendamento referenciados pertençam à mesma clínica.

Nota de implementação: o script é uma base executável para PostgreSQL. A equipe deve validar o mapeamento JPA, os nomes físicos das colunas e a estratégia de geração de UUIDs antes de integrar as migrations à aplicação.

6. Dicionário de dados

O dicionário descreve os campos relevantes, seus tipos, restrições e significado de negócio. Para manter a leitura objetiva, os campos de auditoria padrão (criado_em e atualizado_em) são descritos quando aparecem nas tabelas.

clinica

Campo	Tipo	Restrições	Descrição de negócio
id	UUID	PK, NOT NULL	Identificador da clínica.
nome	VARCHAR(160)	NOT NULL	Nome institucional.
documento	VARCHAR(20)	Opcional	Identificador documental da instituição, se coletado.
email_contato	VARCHAR(254)	Opcional	E-mail institucional para contato.
telefone	VARCHAR(25)	Opcional	Telefone institucional.
status	VARCHAR(15)	CHECK, NOT NULL	Situação da clínica: ATIVA ou INATIVA.
criado_em / atualizado_em	TIMESTAMPTZ	NOT NULL	Datas de criação e última atualização.

usuario

Campo	Tipo	Restrições	Descrição de negócio
id	UUID	PK	Identificador da conta.

clinica_id	UUID	FK, NOT NULL	Clínica à qual a conta está vinculada.
nome	VARCHAR(160)	NOT NULL	Nome de exibição do usuário.
email	VARCHAR(254)	UNIQUE por clínica	Identificador de login.
senha_hash	VARCHAR(255)	NOT NULL	Hash da senha; nunca armazenar senha em texto puro.
telefone	VARCHAR(25)	Opcional	Contato do usuário.
status	VARCHAR(15)	CHECK	ATIVO, INATIVO ou BLOQUEADO.
criado_em / atualizado_em	TIMESTAMPTZ	NOT NULL	Controle temporal da conta.

paciente

Campo	Tipo	Restrições	Descrição de negócio
id	UUID	PK	Identificador do perfil de paciente.
clinica_id	UUID	FK composta	Clínica à qual o paciente pertence.
usuario_id	UUID	FK composta, UNIQUE	Conta de acesso associada ao paciente.
cpf	VARCHAR(14)	Opcional, UNIQUE por clínica	CPF do paciente; dado pessoal usado na identificação cadastral.
data_nascimento	DATE	Opcional	Data de nascimento, dado pessoal.
observacoes_cadastrais	VARCHAR(1000)	Opcional	Observações cadastrais limitadas; não

			usar como prontuário.

profissional

Campo	Tipo	Restrições	Descrição de negócio
id	UUID	PK	Identificador do perfil profissional.
clinica_id	UUID	FK composta	Clínica de vínculo.
usuario_id	UUID	FK composta, UNIQUE	Conta associada ao profissional.
registro_profissional	VARCHAR(40)	Opcional	Número de registro no conselho profissional.
conselho	VARCHAR(40)	Opcional	Conselho profissional correspondente.
ativo	BOOLEAN	NOT NULL	Indica se o profissional está ativo.

especialidade

Campo	Tipo	Restrições	Descrição de negócio
id	UUID	PK	Identificador da especialidade.
clinica_id	UUID	FK, NOT NULL	Clínica responsável pelo cadastro.
nome	VARCHAR(120)	UNIQUE por clínica	Nome da especialidade.
descricao	VARCHAR(500)	Opcional	Descrição da especialidade.
ativa	BOOLEAN	NOT NULL	Indica se a especialidade está disponível.

profissional_especialidade

Campo	Tipo	Restrições	Descrição de negócio

clinica_id	UUID	PK composta, FK	Clínica do relacionamento.
profissional_id	UUID	PK composta, FK	Profissional associado.
especialidade_id	UUID	PK composta, FK	Especialidade associada.

disponibilidade

Campo	Tipo	Restrições	Descrição de negócio
id	UUID	PK	Identificador do intervalo.
clinica_id	UUID	FK composta	Clínica do profissional.
profissional_id	UUID	FK composta	Profissional dono do intervalo.
inicio_em / fim_em	TIMESTAMPTZ	CHECK fim > início	Limites do intervalo de disponibilidade.
status	VARCHAR(15)	CHECK	DISPONIVEL, BLOQUEADO ou AGENDADO.
motivo_bloqueio	VARCHAR(300)	Opcional	Justificativa do bloqueio, quando aplicável.
criado_em	TIMESTAMPTZ	NOT NULL	Data de criação.

agendamento

Campo	Tipo	Restrições	Descrição de negócio
id	UUID	PK	Identificador do agendamento.
clinica_id	UUID	FK composta	Clínica à qual o agendamento pertence.
paciente_id	UUID	FK composta	Paciente atendido.
profissional_id	UUID	FK composta	Profissional responsável.
disponibilidade_id	UUID	FK composta	Intervalo reservado.

inicio_em / fim_em	TIMESTAMPTZ	CHECK fim > início	Período da consulta.
status	VARCHAR(15)	CHECK	AGENDADO, CONFIRMADO, CANCELADO ou REALIZADO.
observacao	VARCHAR(1000)	Opcional	Observação administrativa do agendamento.
criado_por_usuario_id	UUID	FK composta, opcional	Usuário que registrou a marcação, incluindo recepcionista.
criado_em / atualizado_em	TIMESTAMPTZ	NOT NULL	Datas de criação e atualização.

atendimento

Campo	Tipo	Restrições	Descrição de negócio
id	UUID	PK	Identificador do registro de atendimento.
clinica_id	UUID	FK composta	Clínica do atendimento.
agendamento_id	UUID	FK composta, UNIQUE	Agendamento correspondente; no máximo um registro.
status_comparecimento	VARCHAR(12)	CHECK	PRESENTE ou AUSENTE.
registrado_por_usuario_id	UUID	FK composta	Usuário autorizado que registrou o comparecimento.
registrado_em	TIMESTAMPTZ	NOT NULL	Data e hora do registro.
observacao	VARCHAR(1000)	Opcional	Observação administrativa, sem substituir prontuário.

notificacao

Campo	Tipo	Restrições	Descrição de negócio
id	UUID	PK	Identificador da notificação.
clinica_id	UUID	FK	Clínica responsável.
agendamento_id	UUID	FK composta	Agendamento que originou a notificação.
tipo	VARCHAR(15)	CHECK	CONFIRMACAO ou LEMBRETE.
canal	VARCHAR(12)	CHECK	EMAIL ou SISTEMA.
destinatario	VARCHAR(254)	NOT NULL	Destino da notificação.
status	VARCHAR(12)	CHECK	PENDENTE, ENVIADA ou FALHA.
agendada_para / enviada_em	TIMESTAMPTZ	Opcional	Datas de programação e envio.
criado_em	TIMESTAMPTZ	NOT NULL	Data de criação.

papel / permissao / usuario_papel / papel_permissao

Campo	Tipo	Restrições	Descrição de negócio
papel.id / permissao.id	UUID	PK	Identificadores dos papéis e permissões.
papel.nome	VARCHAR(30)	CHECK, UNIQUE por clínica	PACIENTE, PROFISSIONAL, RECEPCIONISTA ou ADMINISTRADOR.
permissao.codigo	VARCHAR(80)	UNIQUE	Código estável da permissão, por exemplo AGENDAMENTO_CANCELAR.
usuario_papel	UUIDs	PK/FKs compostas	Associa contas aos papéis de uma clínica.
papel_permissao	UUIDs	PK/FKs	Associa permissões aos papéis.

registro_auditoria

Campo	Tipo	Restrições	Descrição de negócio

id	UUID	PK	Identificador do evento de auditoria.
clinica_id	UUID	FK	Clínica em cujo contexto ocorreu a operação.
usuario_id	UUID	FK composta, opcional	Usuário responsável pela ação, quando identificado.
acao	VARCHAR(80)	NOT NULL	Operação realizada.
entidade	VARCHAR(80)	NOT NULL	Tipo de registro afetado.
entidade_id	UUID	Opcional	Identificador do registro afetado.
detalhes	JSONB	Opcional	Metadados mínimos da operação; não registrar senhas ou tokens.
criado_em	TIMESTAMPTZ	NOT NULL	Data e hora do evento.

7. Estratégia de dados: backup, LGPD e migrations

7.1 Backup e recuperação

O banco será hospedado no Amazon RDS PostgreSQL em configuração Multi-AZ, conforme a arquitetura. A redundância Multi-AZ apoia a disponibilidade do serviço, mas não substitui backups: exclusões acidentais, corrupção lógica e alterações indevidas exigem mecanismos próprios de recuperação.

Item	Política proposta	Objetivo
Backups automáticos	Diariamente, por meio dos backups automáticos do Amazon RDS.	Permitir recuperação do banco após falha ou erro operacional.
Retenção	Mínimo de 7 dias para backups diários e de 4 semanas para cópias semanais (AWS Backup),	Manter janela suficiente para identificar problemas não

	conforme o RNF007; a janela do point-in-time recovery do RDS inicia em 7 dias e pode ser ampliada após validação de custo	percebidos imediatamente.
Point-in-Time Recovery	Habilitar recuperação para um instante dentro da janela de retenção oferecida pelo RDS.	Reduzir perda de dados em incidentes.
Snapshots	Snapshot manual antes de mudanças estruturais relevantes e releases de maior risco.	Criar ponto de retorno controlado.
Testes de restauração	Executar teste trimestral em ambiente isolado e registrar o resultado.	Verificar se o backup é realmente restaurável.
Acesso aos backups	Restringir a administradores autorizados e registrar operações.	Reduzir risco de acesso indevido.

Metas de recuperação: a arquitetura registra RPO de até 15 minutos e RTO de até 4 horas como objetivos a validar. A política final deverá ser testada na configuração real do RDS e da aplicação; a simples ativação de backup ou Multi-AZ não comprova, isoladamente, o atendimento dessas metas.

7.2 LGPD e proteção de dados

O VittaCare tratará dados pessoais e poderá tratar dados pessoais sensíveis relacionados à saúde. A aplicação dos princípios da LGPD deverá ser incorporada desde o desenho do banco, com coleta mínima, finalidade definida, acesso restrito, segurança e retenção controlada.

Campo / grupo	Classificação	Cuidados previstos
usuario.nome, usuario.email, usuario.telefone	Dado pessoal	Acesso conforme perfil; exibição somente quando necessária; proteção em trânsito.
paciente.data_nascimento	Dado pessoal	Coleta somente quando necessária ao processo da

		clínica; restringir visualização.
paciente.cpf	Dado pessoal	Coleta somente quando necessária ao cadastro; acesso restrito por perfil e por clínica; mascarar na exibição quando possível; nunca registrar em logs.
profissional.registro_profissional e conselho	Dado pessoal profissional	Acesso administrativo e profissional conforme necessidade.
agendamento.inicio_em, fim_em e status	Dado pessoal quando associado ao paciente	Restringir por clínica e por perfil; evitar exposição pública.
atendimento.status_comparecimento	Dado pessoal associado a serviço de saúde	Acesso restrito; não ampliar para conteúdo clínico fora do escopo.
usuario.senha_hash	Credencial protegida	Armazenar somente hash seguro, preferencialmente Argon2id ou BCrypt; nunca registrar em logs.
registro_auditoria.detalhes	Metadados potencialmente pessoais	Registrar somente o necessário; excluir senhas, tokens e conteúdo sensível desnecessário.

- Aplicar autorização no back-end em 100% dos endpoints que exponham informações restritas.
- Aplicar filtro por clinica_id em consultas, inserções, atualizações e exclusões.
- Obter o contexto da clínica a partir da identidade autenticada e das associações autorizadas.
- Não utilizar o campo de observação como substituto de prontuário eletrônico.

- Definir política de retenção e descarte seguro de acordo com a finalidade e as obrigações legais aplicáveis.
- Manter trilha de auditoria para operações administrativas críticas, sem armazenar credenciais ou tokens.
- Restringir acesso ao banco a partir das instâncias autorizadas, em sub-redes privadas, conforme a arquitetura.

A definição da base legal, dos prazos legais de retenção e dos procedimentos para atendimento aos direitos dos titulares deverá ser validada pela instituição responsável e por orientação jurídica/encarregado de dados. Este documento estabelece controles técnicos propostos, não substitui uma avaliação jurídica.

7.3 Migrations e evolução do esquema

As alterações do esquema serão versionadas por migrations, preferencialmente com Flyway integrado ao Spring Boot. Cada migration deverá ser imutável após aplicação em ambiente compartilhado, possuir identificador sequencial e ser revisada junto ao código que depende dela.

Etapa	Procedimento
1. Desenvolvimento	Criar migration versionada e testar em banco local com a mesma versão principal do PostgreSQL.
2. Validação	Executar testes de integração, constraints, índices e consultas críticas.
3. Staging	Aplicar migration em ambiente de homologação com dados sintéticos ou anonimizados.
4. Produção	Aplicar mudanças compatíveis antes de publicar código que dependa delas.
5. Monitoramento	Verificar duração, locks, erros, integridade e métricas após a aplicação.

7.4 Estratégia para migrations sem downtime

1. Expandir: adicionar coluna ou tabela nova como opcional, sem remover ou renomear imediatamente estruturas utilizadas pela versão atual.
2. Compatibilizar: publicar código que consiga ler o formato antigo e o novo, quando necessário.
3. Migrar: preencher dados em lotes pequenos, monitorando carga e bloqueios.
4. Trocar: direcionar a aplicação para a nova estrutura após validar consistência.

5. Contrair: remover coluna ou estrutura antiga somente em uma migration posterior, quando nenhuma versão ativa depender dela.

Índices grandes deverão ser planejados para criação concorrente quando aplicável no PostgreSQL. Operações que exigem locks longos devem ser agendadas e testadas previamente. Uma migration não deve ser considerada sem downtime apenas por estar versionada: o impacto real depende do comando, do volume de dados e da estratégia de implantação.

8. Regras de integridade e rastreabilidade

Regra	Implementação no banco / aplicação	Requisitos relacionados
Autenticação	E-mail único por clínica, hash de senha e status de conta; validação de credenciais no Spring Security.	RF001, RF002, RF003
Cadastro de paciente	Tabela paciente vinculada a usuario e clínica por FK composta.	RF005, RF006
Profissionais e especialidades	Tabelas profissional, especialidade e associação N:N.	RF007, RF008, RF009
Pesquisa e disponibilidade	Índices por clínica, profissional e início do intervalo.	RF010, RF011, RF012, RF013
Agendamento	FKs para paciente, profissional e disponibilidade; transação no serviço.	RF014, RF015, RF018
Cancelamento e remarcação	Atualização controlada de status e criação de nova reserva na mesma unidade transacional quando aplicável.	RF016, RF017
Prevenção de duplicidade	Índice único parcial para uma reserva ativa por disponibilidade.	RF018
Histórico	Índices para consulta por paciente e profissional.	RF019
Comparecimento	Tabela atendimento com UNIQUE por agendamento.	RF020

Notificações	Tabela notificacao vinculada ao agendamento.	RF021, RF022
Relatórios	Consultas agregadas sobre agendamentos, cancelamentos e ausências.	RF023
Permissões	Papéis, permissões e associações; autorização efetiva no back-end.	RF024
Isolamento multi-tenant	clinica_id nas tabelas e FKs compostas para impedir referências entre clínicas.	RF025
Bloqueio de horário	Validação transacional no serviço antes de alterar disponibilidade; não bloquear horário com agendamento ativo.	RF026

8.1 Pontos que exigem validação na implementação

- Definir como o paciente seleciona a clínica no autorregistro, por exemplo por endereço específico ou convite, sem confiar em clinica_id arbitrário enviado pelo navegador.
- Definir política de normalização do e-mail (por exemplo, armazenar em minúsculas) antes de aplicar a restrição de unicidade.
- Definir a granularidade das disponibilidades: intervalos cadastrados pelo profissional ou slots derivados de uma agenda. O script representa intervalos reserváveis individualmente.
- Garantir que o horário do agendamento corresponda ao intervalo da disponibilidade e que o profissional do agendamento seja o mesmo profissional dono da disponibilidade. Essa consistência deverá ser garantida por validação no serviço e pode ser reforçada com constraints adicionais após a decisão final sobre a granularidade.
- Definir catálogo inicial de permissões e papéis, com carga de dados de referência por migration.
- Definir processo de anonimização ou descarte de dados quando houver solicitação válida e não existir obrigação de retenção.

9. Considerações finais

A modelagem proposta mantém a escolha tecnológica do Bloco 3 e traduz as entidades e regras dos Blocos 1, 2 e 4 para um banco relacional PostgreSQL. O uso de chaves estrangeiras, constraints, índices e transações oferece uma base consistente para os fluxos centrais do VittaCare, especialmente o agendamento e o isolamento lógico entre clínicas.

O modelo físico apresentado é executável em PostgreSQL e fornece uma estrutura inicial para implementação. Antes da entrada em produção, deverão ser concluídos os testes de concorrência, validação do mapeamento JPA, revisão das permissões, definição das políticas de retenção e testes reais de backup e recuperação.

Dessa forma, o Bloco 5 entrega a passagem do modelo conceitual para uma estrutura lógica e física que pode orientar a implementação do banco de dados do VittaCare.